

Contents

Week 4: Feature Selection & Classical Machine Learning	2
AI/ML in Biomarker Discovery — miRNA in Alzheimer’s Disease	2
Learning Objectives	2
Conceptual Overview: Why We Need Both Differential Expression and Machine Learning	2
MODULE 4.1 — Differential Expression Analysis with limma-voom	3
4.1.1 When to Use limma vs DESeq2	3
4.1.2 The limma-voom Pipeline for Microarray	3
4.1.3 Understanding the Design Matrix	5
4.1.4 Multiple Testing Correction: Benjamini-Hochberg FDR	5
4.1.5 Interpreting Fold Changes: What Does logFC = 1.0 Mean?	6
4.1.6 Volcano Plot and MA Plot	6
4.1.7 MCI vs Control, AD vs MCI and the Three-Way Summary	8
MODULE 4.2 — Differential Expression Analysis with DESeq2	9
4.2.1 The DESeq2 Workflow for RNA-seq	9
4.2.2 Log Fold Change Shrinkage with lfcShrink	10
4.2.3 Understanding the DESeq2 Results Table	11
4.2.4 Contrasts That Are Not Coefficients: type = “ashr”	11
4.2.5 Cross-Dataset Overlap: Does the Signal Replicate?	11
MODULE 4.3 — Feature Selection Methods	12
4.3.1 The Dimensionality Problem in Machine Learning	12
4.3.2 Three Classes of Feature Selection	12
4.3.3 Filter Methods: Univariate Statistical Tests	13
4.3.4 Wrapper Methods: Recursive Feature Elimination (RFE)	14
4.3.5 Embedded Methods: LASSO Regularization	14
4.3.6 Embedded Methods: Random Forest Feature Importance	15
4.3.7 Choosing Among Feature Selection Methods	15
MODULE 4.4 — Building a Logistic Regression Classifier	16
4.4.1 What is Logistic Regression?	16
4.4.2 Train/Test Split: The Foundation of Honest Evaluation	16
4.4.3 Fitting Logistic Regression and Interpreting Coefficients	17
4.4.4 L1 vs L2 Regularization	18
MODULE 4.5 — Support Vector Machine (SVM)	19
4.5.1 The Maximal Margin Intuition	19
4.5.2 The Kernel Trick for Non-Linear Separation	19
4.5.3 Cross-Validated Grid Search for SVM Hyperparameters	19
4.5.4 Support Vectors and Their Biological Meaning	20
MODULE 4.6 — Random Forest	20
4.6.1 The Ensemble Principle	20
4.6.2 Gini Impurity and Node Splitting	21
4.6.3 Feature Importance via Mean Decrease in Gini Impurity	21
4.6.4 Out-of-Bag (OOB) Error	22
4.6.5 Choosing the Number of Trees	22
MODULE 4.7 — Model Evaluation	23
4.7.1 The Confusion Matrix	23
4.7.2 Clinical Metrics from the Confusion Matrix	24
4.7.3 ROC Curve and AUC	25
4.7.4 Why AUC Alone Is Insufficient: The Imbalanced Class Problem	25
4.7.5 Cross-Validation for Unbiased AUC Estimates	26
4.7.6 Model Calibration	27
BIOLOGICAL INTERPRETATION CALLOUT: When miR-29b is the Top Feature	28
WEEK 4 LAB SESSIONS	28
Lab 4A: Differential Expression Analysis in R (90 min)	28
Lab 4B: ML Classifier in Python (90 min)	29
WEEK 4 ASSIGNMENTS	29

Required Reading	29
Reflection Questions (Discuss in Week 5 opening session)	29
Practical Exercise	30
WEEK 4 GLOSSARY	30
KEY REFERENCES (Week 4)	32
NEXT WEEK PREVIEW: Advanced ML & Validation (Week 5)	32

Week 4: Feature Selection & Classical Machine Learning

AI/ML in Biomarker Discovery — miRNA in Alzheimer’s Disease

Learning Objectives

By the end of Week 4, you will be able to: 1. Run a complete differential expression analysis using limma (microarray, GSE120584) and DESeq2 (RNA-seq, GSE46579), correctly interpreting log2 fold changes, adjusted p-values, and shrinkage estimators in the context of blood miRNA biology 2. Explain the dimensionality problem in machine learning and articulate why feature selection is mandatory before training a classifier on a dataset with hundreds of miRNAs — especially in typical cohorts of 100–200 samples such as our validation set GSE46579 (65 samples) 3. Apply and compare three classes of feature selection methods — filter, wrapper, and embedded — and justify your choice of method for a given dataset 4. Build, train, and evaluate logistic regression, SVM, and random forest classifiers for AD vs Control classification using scikit-learn, including proper train/test splitting and cross-validation 5. Interpret model outputs in biological terms: what does it mean when miR-29b is the top feature in a random forest model, and how would you validate that finding experimentally? 6. Evaluate classifier performance using the full suite of clinically relevant metrics: sensitivity, specificity, PPV, NPV, AUC, and precision-recall curves — and explain why AUC alone is insufficient for imbalanced clinical datasets 7. Identify and avoid the most common overfitting mistakes made when building biomarker ML models on small genomic datasets

Conceptual Overview: Why We Need Both Differential Expression and Machine Learning

A common question from wet-lab biologists entering this field: *Why do we need machine learning at all? If certain miRNAs are differentially expressed between AD and control, why not just use those?*

The answer involves three levels of complexity.

Level 1 — Single marker insufficiency. No individual miRNA is changed exclusively in AD. Every candidate miRNA is also influenced by age, sex, medications, co-morbidities, and blood collection technique. A single miRNA threshold cannot reliably distinguish AD from control in a real clinical population. The Ludwig et al. (2019) study, which validated 21 miRNAs with RT-qPCR in 465 individuals, achieved only AUC = 87.6% — a clinically meaningful but imperfect performance. A *panel* of miRNAs, learned jointly by a classifier, consistently outperforms any single marker.

Level 2 — Non-linearity and interaction. Two miRNAs might individually show modest differences between groups, but their *combination* (ratio, product, co-expression pattern) might be highly discriminative. Linear differential expression tests miss these combinatorial signals. Machine learning algorithms capture them.

Level 3 — Dimensionality reduction for clinical translation. A validated biomarker panel needs to be translatable to a clinical RT-qPCR assay. Differential expression narrows 500 profiled miRNAs down to dozens of candidates; ML feature selection and embedded methods further

narrow those dozens to the smallest panel that maintains discriminative power. Zhao et al. (2020) achieved 85.7% accuracy with just 12 miRNAs — a panel small enough for a practical clinical test. The workflow this week follows this logic: **differential expression → feature ranking → ML training → evaluation**. The R script (Week4_DE_FeatureSelection.R) covers the differential expression and feature ranking steps; the Python Jupyter notebook (Week4_ML_Classifier.ipynb) covers the ML steps.

MODULE 4.1 — Differential Expression Analysis with limma-voom

4.1.1 When to Use limma vs DESeq2

For historical reasons, two complementary methods dominate miRNA differential expression analysis:

Method	Data Type	Statistical Model	When to Use
DESeq2	RNA-seq count data	Negative binomial generalized linear model	GSE46579 (small RNA-seq)
limma-voom	Microarray <i>or</i> RNA-seq	Linear model with precision weighting	GSE120584 (microarray; limma without voom); also valid for RNA-seq with large N
edgeR	RNA-seq count data	Negative binomial (quasi-likelihood or exact test)	Alternative to DESeq2; especially good for very small N

Both approaches produce the same essential outputs: a table of miRNAs ranked by evidence for differential expression, with fold changes and multiple-testing-corrected p-values. The difference is in the underlying statistical assumptions and how they handle mean-variance relationships in the data.

4.1.2 The limma-voom Pipeline for Microarray

limma (Linear Models for Microarray Analysis, Ritchie et al. 2015) was originally designed for microarray data. The **voom** transformation extends it to RNA-seq count data by: 1. Converting counts to log2-CPM values 2. Estimating the mean-variance trend across all features 3. Computing precision weights for each observation that downweight noisy (low-count) features 4. Fitting weighted linear models to the precision-weighted data

For our purposes with **GSE120584** (Toray 3D-Gene microarray, already log2-transformed and normalized by the submitters; Week 2), we use limma without the voom step — the data is already on the appropriate scale. We fit all three groups at once and adjust for sex and age, because Week 2 and Week 3 showed that AD patients are older and more often female than Controls.

```
# =====
# limma pipeline for GSE120584 (microarray, primary dataset)
# =====

library(limma)

# Load the normalized log2 matrix and metadata (output from Week 2)
expr_clean <- readRDS("data/processed/GSE120584_expr_clean.rds") # 925 miRNAs × 1328 samples
```

```

metadata <- readRDS("data/processed/GSE120584_metadata_clean.rds")

# Step 1: Set up the design matrix
# Each row = one sample; each column = one coefficient to estimate.
# ~ 0 + group gives one coefficient per group (no intercept), which makes
# contrasts easy to write. Group labels contain spaces and an apostrophe,
# so we use short names inside the design.
short_names <- c("Control" = "Control",
                 "Mild Cognitive Impairment" = "MCI",
                 "Alzheimer's Disease" = "AD")
metadata$group_short <- factor(short_names[as.character(metadata$group)],
                              levels = c("Control", "MCI", "AD"))

design <- model.matrix(~ 0 + group_short + sex + age, data = metadata)
colnames(design) <- sub("^group_short", "", colnames(design))
colnames(design) # "Control" "MCI" "AD" "sexmale" "age"

# Step 2: Fit linear models to each miRNA
# lmFit estimates the coefficients (group means, sex and age effects)
# for every miRNA simultaneously.
fit <- lmFit(expr_clean, design)

# Step 3: Define contrasts – the comparisons of interest
contrast_matrix <- makeContrasts(
  AD_vs_Control = AD - Control,
  MCI_vs_Control = MCI - Control,
  AD_vs_MCI = AD - MCI,
  levels = design
)
fit2 <- contrasts.fit(fit, contrast_matrix)

# Step 4: Empirical Bayes moderation
# eBayes "borrows strength" across all miRNAs to better estimate each miRNA's
# variance: each miRNA's variance is shrunk toward a common prior.
# This moderation is why limma outperforms simple t-tests for genomics.
fit2 <- eBayes(fit2, trend = TRUE) # trend=TRUE: variance may depend on expression level

# Step 5: Extract results table for one contrast
results_limma <- topTable(
  fit2,
  coef = "AD_vs_Control",
  number = Inf, # return ALL miRNAs, not just top 10
  adjust = "BH", # Benjamini-Hochberg FDR correction
  sort.by = "P"
)

# The results table contains:
# logFC -- log2 fold change (AD relative to Control)
# AveExpr -- average log2 expression across all samples
# t -- moderated t-statistic
# P.Value -- raw p-value
# adj.P.Val -- Benjamini-Hochberg FDR-adjusted p-value
# B -- log-odds of differential expression (B > 0 means more likely DE)

lfc_cut <- 0.3 # see Section 4.1.5 for why 0.3 on this array
sig_AD <- results_limma[results_limma$adj.P.Val < 0.05 & abs(results_limma$logFC) > lfc_cut, ]

```

```
cat("Significant (FDR < 0.05, |logFC| > 0.3):", nrow(sig_AD), "\n") # 49
cat(" Up in AD:", sum(sig_AD$logFC > 0), " Down in AD:", sum(sig_AD$logFC < 0), "\n") # 45 / 4
```

4.1.3 Understanding the Design Matrix

The design matrix is one of the most conceptually challenging parts of limma for biologists. Here is an intuition:

Sample	Control	AD	
-----	-----	--	
GSM_ctrl_001	1	0	<- this sample contributes to the Control coefficient
GSM_ctrl_002	1	0	
GSM_ctrl_003	1	0	
GSM_ad_001	0	1	<- this sample contributes to the AD coefficient
GSM_ad_002	0	1	
GSM_ad_003	0	1	

The linear model for each miRNA fits: $\text{expression} = \text{beta_Control} * (\text{is_control}) + \text{beta_AD} * (\text{is_AD}) + \text{error}$

The contrast AD - Control then computes $\text{beta_AD} - \text{beta_Control}$ for each miRNA, which is exactly the log2 fold change between groups.

Including covariates: If your metadata includes age and sex, you can include them as covariates to increase statistical power and control for confounding:

```
# Design with covariates – what the Week 4 script uses for GSE120584
design_cov <- model.matrix(~ 0 + group_short + sex + age, data = metadata)
# The contrasts remain the same; limma estimates the group effects
# after accounting for age and sex
```

4.1.4 Multiple Testing Correction: Benjamini-Hochberg FDR

When you test 500 miRNAs simultaneously, random chance alone will produce approximately 25 miRNAs with $p < 0.05$ even if none are truly differentially expressed ($5\% \times 500 = 25$ false positives). Multiple testing correction adjusts p-values to control the **False Discovery Rate (FDR)** — the expected proportion of your “significant” results that are actually false.

Benjamini-Hochberg (BH) procedure:

1. Rank all p-values from smallest to largest: $p_1 \leq p_2 \leq \dots \leq p_m$
2. For a desired FDR level q (typically 0.05), find the largest k such that $p_k \leq (k/m) \times q$
3. Reject hypotheses 1 through k

This guarantees that among all miRNAs called significant, at most 5% are expected to be false positives.

Threshold	Meaning
adj.P.Val < 0.05	At most 5% expected false positives among called DE miRNAs
adj.P.Val < 0.10	At most 10% expected false positives — more lenient, for discovery
P.Value < 0.05	Uncorrected; expect ~5% of ALL tested miRNAs to be false positives

A common mistake: Students often filter by raw p-value and miss the correction. In a 500-miRNA study, an uncorrected p-value of 0.05 is essentially meaningless. Always report and filter on adj .P.Val.

4.1.5 Interpreting Fold Changes: What Does logFC = 1.0 Mean?

The logFC column in limma and DESeq2 results is expressed in **log2 scale**. This is a deliberately chosen scale because: - It makes upregulation and downregulation symmetric: logFC of +2 (4-fold up) and -2 (4-fold down) are equally “extreme” - It approximately follows a normal distribution (amenable to t-statistics) - It compresses the wide dynamic range of miRNA expression

Converting log2 fold change to biological fold change:

logFC	Fold Change	Biological Interpretation
0.0	1.0x	No change
0.5	1.41x	41% increase
1.0	2.0x	Doubled in AD blood
1.5	2.83x	Nearly tripled
2.0	4.0x	Quadrupled
-1.0	0.5x	Halved in AD blood
-2.0	0.25x	Only 25% of control level remains

What does logFC = 1.0 mean for a blood miRNA biomarker?

A logFC of 1.0 for a serum miRNA in AD means the **average circulating level is twice as high in AD patients compared to controls** (after normalization). In a blood-based biomarker context, this is a clinically meaningful difference — if you performed RT-qPCR validation on independent samples, you would expect to see approximately a 2-cycle difference in Ct values (since each 2x change in expression corresponds to approximately 1 Ct unit difference in log2 terms, or a 1-Ct change in standard qPCR reporting).

However, fold change alone does not determine clinical utility. A miRNA with logFC = 2.0 but very high variance within each group might be completely useless as a biomarker, while a miRNA with logFC = 0.8 and extremely low within-group variance could be highly discriminative. This is why ML models — which model the full distribution, not just group means — often identify better biomarkers than simple fold-change rankings.

Fold changes on a microarray are compressed. Background fluorescence, cross-hybridization and a limited dynamic range make array fold changes smaller than the true biological differences — and smaller than RNA-seq fold changes for the same biology. In GSE120584, **449 of 925 miRNAs pass FDR < 0.05 for AD vs Control, yet the largest |log2FC| is only 0.43** (1.35-fold). The usual RNA-seq cut-off of $|\log_2\text{FC}| > 0.5$ (or 1) would keep nothing. At the same time, with > 1,000 samples even tiny differences become “significant”, so a fold-change cut-off is still needed. The Week 4 script therefore uses $|\log_2\text{FC}| > 0.3$ (≈ 1.23 -fold) for GSE120584 and 0.5 for the RNA-seq dataset GSE46579. Always choose the cut-off with the platform in mind — and report it.

4.1.6 Volcano Plot and MA Plot

Volcano plot: Visualizes both fold change (x-axis) and statistical significance (y-axis, as $-\log_{10}(\text{p-value})$) for every miRNA simultaneously. Ideal for identifying the most biologically and statistically interesting candidates.

```

library(ggplot2)
library(ggrepel)

# Prepare data frame for plotting
volcano_df <- results_limma
volcano_df$miRNA <- rownames(volcano_df)
volcano_df$significance <- "Not Significant"
volcano_df$significance[volcano_df$adj.P.Val < 0.05 & volcano_df$logFC > 0.3] <- "Up in AD"
volcano_df$significance[volcano_df$adj.P.Val < 0.05 & volcano_df$logFC < -0.3] <- "Down in AD"
volcano_df$significance <- factor(volcano_df$significance,
                                levels = c("Not Significant", "Up in AD", "Down in AD"))

# Identify top 15 miRNAs to label (by smallest FDR)
top15 <- head(volcano_df[order(volcano_df$adj.P.Val), ], 15)

p_volcano <- ggplot(volcano_df,
                    aes(x = logFC, y = -log10(P.Value), colour = significance)) +
  geom_point(alpha = 0.6, size = 1.5) +
  geom_point(data = top15, size = 2.5, alpha = 0.9) +
  geom_vline(xintercept = c(-0.3, 0.3), linetype = "dashed",
             colour = "grey40", linewidth = 0.4) +
  geom_hline(yintercept = -log10(0.05), linetype = "dashed",
             colour = "grey40", linewidth = 0.4) +
  geom_text_repel(
    data = top15,
    aes(label = miRNA),
    size = 2.8,
    max.overlaps = 20,
    box.padding = 0.4,
    colour = "black"
  ) +
  scale_colour_manual(
    values = c("Not Significant" = "grey70",
              "Up in AD" = "#D73027",
              "Down in AD" = "#4575B4")
  ) +
  labs(
    title = "Volcano Plot: AD vs Control (GSE120584, limma)",
    x = "log2 Fold Change (AD / Control)",
    y = "-log10(P-value)",
    colour = NULL,
    caption = "Dashed lines: |logFC| > 0.3 and p < 0.05 (uncorrected)"
  ) +
  theme_bw(base_size = 12) +
  theme(
    plot.title = element_text(face = "bold"),
    legend.position = "top"
  )

print(p_volcano)
ggsave("results/volcano_limma_AD_vs_Control.png", p_volcano, width = 8, height = 6, dpi = 150)

```

MA plot: Shows mean expression on the y-axis (M, for Mean difference) and average expression (A, for Average) on the x-axis. Useful for identifying expression-level-dependent biases in fold changes.

For limma results, construct MA plot manually

```

ma_df <- volcano_df

p_ma <- ggplot(ma_df, aes(x = AveExpr, y = logFC, colour = significance)) +
  geom_point(alpha = 0.5, size = 1.2) +
  geom_hline(yintercept = 0, colour = "black", linewidth = 0.5) +
  geom_hline(yintercept = c(-0.3, 0.3), linetype = "dashed",
             colour = "grey40", linewidth = 0.4) +
  scale_colour_manual(
    values = c("Not Significant" = "grey70",
              "Up in AD"         = "#D73027",
              "Down in AD"       = "#4575B4")
  ) +
  labs(
    title = "MA Plot: AD vs Control (GSE120584, limma)",
    x     = "Average log2 Expression (A)",
    y     = "log2 Fold Change (M)",
    colour = NULL
  ) +
  theme_bw(base_size = 12) +
  theme(plot.title = element_text(face = "bold"), legend.position = "top")

print(p_ma)
ggsave("results/ma_plot_limma_AD_vs_Control.png", p_ma, width = 8, height = 5, dpi = 150)

```

Interpreting the MA plot: In a well-normalized dataset, the cloud of points should be centered around $M = 0$ across the full range of expression levels (A-axis). If low-expression miRNAs show a systematic upward or downward trend, this indicates a normalization issue. Most DE miRNAs (colored points) should be scattered at various expression levels — if they cluster only at high expression levels, you may be missing DE miRNAs at low expression due to insufficient statistical power.

What the GSE120584 volcano plot shows: it is strongly asymmetric — 45 of the 49 significant miRNAs are **up** in AD, most of them low-abundance miRNAs close to the detection floor (many poorly characterised miR-6xxx-3p species; the top hit is hsa-miR-6761-3p, $\log_2FC +0.43$). None of the Week 1 “classic” AD miRNAs is among them. Because Week 2 showed that AD samples detect more miRNAs above background than Controls, part of this signal may be technical. A lopsided volcano plot is a prompt to look for a technical explanation before a biological one.

4.1.7 MCI vs Control, AD vs MCI and the Three-Way Summary

All three comparisons come from the same limma fit — just ask `topTable()` for a different contrast:

```

res_MCI    <- topTable(fit2, coef = "MCI_vs_Control", number = Inf, sort.by = "P")
res_AD_MCI <- topTable(fit2, coef = "AD_vs_MCI",          number = Inf, sort.by = "P")

sig_MCI    <- res_MCI[res_MCI$adj.P.Val < 0.05 & abs(res_MCI$logFC) > lfc_cut, ]
sig_AD_MCI <- res_AD_MCI[res_AD_MCI$adj.P.Val < 0.05 & abs(res_AD_MCI$logFC) > lfc_cut, ]
nrow(sig_MCI)      # 44 (all up in MCI)
nrow(sig_AD_MCI)   # 0

# Progressive markers: significant in both, same direction
common    <- intersect(rownames(sig_AD), rownames(sig_MCI))
progressive <- common[sign(sig_AD[common, "logFC"]) == sign(sig_MCI[common, "logFC"])]
length(progressive) # 12

```

Understanding which miRNAs change at each disease stage is biologically crucial:

- **MCI vs Control only:** Potential *early detection* biomarkers — change before dementia onset
- **AD vs Control only:** *Disease-stage* markers — reflect advanced pathology
- **MCI vs Control AND AD vs Control (same direction):** *Progressive* markers — change early and worsen
- **AD vs MCI:** Markers that distinguish conversion from MCI to AD — potential *progression* monitoring markers

DISEASE CONTINUUM:

```
Normal -> [MCI vs Control changes] -> MCI -> [AD vs MCI changes] -> AD
<----->
                AD vs Control (combined signal)
```

Biologically, miRNAs that change progressively across all three stages (Control -> MCI -> AD, monotonically) are the most compelling biomarker candidates. These “stepwise” miRNAs reflect continuous biological deterioration rather than a single threshold event.

Power matters: GSE120584 has only 32 MCI samples. “Late” markers — significant in AD vs Control but not in MCI vs Control — mostly show the *same* direction and nearly the same size of change in MCI (e.g. miR-3184-5p: log2FC -0.32 in AD, -0.28 in MCI, FDR 0.056). “Not significant in MCI” is absence of evidence, not evidence of absence. Together with zero AD vs MCI differences, the data fit an early shared change better than a stepwise progression.

MODULE 4.2 — Differential Expression Analysis with DESeq2

4.2.1 The DESeq2 Workflow for RNA-seq

DESeq2 (Love, Huber, and Anders, 2014) is the gold standard for RNA-seq differential expression analysis. It models count data with a **negative binomial distribution** — appropriate for miRNA-seq because counts are overdispersed (variance exceeds the mean, violating Poisson assumptions).

The three key innovations of DESeq2 are: 1. **Median-of-ratios normalization** — robust size factor estimation (covered in Week 2) 2. **Empirical Bayes dispersion estimation** — shares dispersion information across miRNAs to stabilize estimates from small N 3. **Log fold change shrinkage (lfcShrink)** — pulls extreme fold changes from noisy, low-count miRNAs toward zero, producing more reliable estimates

In this course DESeq2 is used for the **RNA-seq validation dataset, GSE46579** (whole blood; AD vs Control — this cohort has no MCI group). DESeq2 needs **raw counts**, never VST or log values: it normalizes internally.

```
# =====
# DESeq2 differential expression -- GSE46579 (small RNA-seq)
# One comparison: AD vs Control
# =====
```

```
library(DESeq2)
```

```
# Load objects from Week 2
```

```
counts_46 <- readRDS("data/processed/GSE46579_counts_filtered.rds") # 273 miRNAs × 65 samples
meta_46 <- readRDS("data/processed/GSE46579_metadata_clean.rds")
```

```
meta_46$group <- factor(meta_46$group, levels = c("Control", "Alzheimer's Disease"))
```

```
meta_46$sex <- factor(meta_46$sex)
```

```
meta_46$age_scaled <- as.numeric(scale(meta_46$age)) # numeric covariates fit best on a small s
```

```

dds_46 <- DESeqDataSetFromMatrix(
  countData = counts_46,
  colData   = meta_46,
  design    = ~ sex + age_scaled + group
)

# Relevel: Control is the reference level
dds_46$group <- relevel(dds_46$group, ref = "Control")

# Run DESeq2 (estimates size factors, dispersions, and fits the GLM)
dds_46 <- DESeq(dds_46)

# Check what coefficients were estimated
resultsNames(dds_46)
# [1] "Intercept" "sex_male_vs_female" "age_scaled"
# [4] "group_Alzheimer.s.Disease_vs_Control"

```

4.2.2 Log Fold Change Shrinkage with lfcShrink

A critical step that many published studies omit: **shrinkage of log fold change estimates**.

Without shrinkage, miRNAs with very few counts show extreme (but unreliable) fold changes. For example, a miRNA with 1 count in one group and 3 counts in another group looks like a 3-fold change — but this is driven by noise, not biology.

lfcShrink uses an empirical Bayes approach (apeglm method) to shrink these unreliable fold changes toward zero while leaving well-estimated, high-count fold changes largely unchanged.

```

# ---- Comparison 1: AD vs Control ----
res_46 <- lfcShrink(
  dds_46,
  coef = "group_Alzheimer.s.Disease_vs_Control",
  type = "apeglm" # apeglm: best shrinkage method (Zhu et al. 2019)
)

# Convert to data frame and sort by adjusted p-value
res_AD_df <- as.data.frame(res_46)
res_AD_df$miRNA <- rownames(res_AD_df)
res_AD_df <- res_AD_df[order(res_AD_df$padj, na.last = TRUE), ]

# Filter to significant results
sig_AD <- res_AD_df[!is.na(res_AD_df$padj) &
  res_AD_df$padj < 0.05 &
  abs(res_AD_df$log2FoldChange) > 0.5, ]

cat("=== AD vs Control (GSE46579, DESeq2) ===\n")
cat("Total miRNAs tested:", nrow(res_AD_df), "\n")
cat("Significant (FDR < 0.05, |log2FC| > 0.5):", nrow(sig_AD), "\n")
cat("  Upregulated in AD:", sum(sig_AD$log2FoldChange > 0), "\n")
cat("  Downregulated in AD:", sum(sig_AD$log2FoldChange < 0), "\n")
print(head(sig_AD[, c("miRNA", "log2FoldChange", "lfcSE", "pvalue", "padj")], 15))

```

Result for GSE46579: 273 miRNAs tested; 105 pass FDR < 0.05, and 77 pass FDR < 0.05 and |log2FC| > 0.5 (30 up, 47 down in AD). The top hits — hsa-let-7f-5p and hsa-miR-98, both down in AD — have log2FC around -0.9 to -1.0: RNA-seq fold changes are much larger than the array's.

If `apegglm` is not installed, the Week 4 script falls back to `type = "normal"`; install it with `BiocManager::install("apeglm")`.

4.2.3 Understanding the DESeq2 Results Table

Column	Meaning	Interpretation
baseMean	Average normalized count across all samples	Expression level; low values indicate noisy estimates
log2FoldChange	Shrunk $\log_2(\text{AD}/\text{Control})$	Effect size; positive = higher in AD
lfcSE	Standard error of the shrunk fold change	Precision of the estimate
stat	Wald test statistic	Fold change / standard error
pvalue	Wald test p-value	Uncorrected probability
padj	Benjamini-Hochberg adjusted p-value	FDR-corrected; use this for filtering

Independent filtering: DESeq2 automatically performs **independent filtering** — it removes miRNAs with very low mean expression (where statistical power is essentially zero) before adjusting p-values. This is not bias; it is a mathematically sound step that improves power for the miRNAs that *can* be tested. The `padj` column shows NA for miRNAs that were excluded by independent filtering.

4.2.4 Contrasts That Are Not Coefficients: `type = "ashr"`

GSE46579 has only two groups, so AD vs Control is a single named coefficient and `type = "apeglm"` works. In a DESeq2 design with **three** groups and Control as the reference (e.g., a sequencing cohort with an MCI group), the named coefficients would be `MCI_vs_Control` and `AD_vs_Control` only. AD vs MCI is then a *difference* of coefficients, expressed as a contrast — and `apeglm` cannot shrink it:

```
# Illustration for a three-group count dataset (not needed for GSE46579)
res_AD_vs_MCI <- lfcShrink(
  dds,
  contrast = c("group", "Alzheimer's Disease", "Mild Cognitive Impairment"),
  type      = "ashr"           # ashr works with arbitrary contrasts
)
```

(Alternatively, re-level the factor so MCI is the reference and use `apeglm` on the new coefficient.) In GSE120584 the three-group comparisons are done with `limma` (Section 4.1.7), which needs no fold-change shrinkage: `limma` models continuous $\log_2$ values and its `eBayes()` step moderates the **variances** instead.

4.2.5 Cross-Dataset Overlap: Does the Signal Replicate?

miRNAs that are significant in **both** the serum microarray (GSE120584, `limma`) and the whole-blood RNA-seq (GSE46579, DESeq2) are far more trustworthy than those significant in only one — dataset-specific findings can reflect platform artefacts, cohort-specific confounding, or batch effects.

```
overlap <- intersect(rownames(sig_AD), sig_46$miRNA) # names must match exactly here
```

Result: only **69** miRNA names are tested in both datasets (GSE46579 uses miRBase v18 names, e.g. hsa-miR-98 instead of hsa-miR-98-5p; Week 5 fixes this with miRBaseConverter), and only **1** miRNA — hsa-miR-4435, up in AD in both — is significant in both lists.

Why so little overlap? - **Naming:** different miRBase versions hide shared miRNAs (Week 5) - **Platform:** array and sequencing detect different miRNAs with different sensitivity; many GSE120584 hits are barely detected by RNA-seq - **Sample type:** serum is cell-free; whole blood is dominated by red- and white-cell miRNAs - **Cohort and power:** different populations, age and sex structures; only 65 samples in GSE46579

A high overlap with **low direction concordance** would additionally suggest a labelling error or a confounder acting differently in each cohort.

MODULE 4.3 — Feature Selection Methods

4.3.1 The Dimensionality Problem in Machine Learning

Consider the central challenge of miRNA biomarker datasets: many published cohorts have around **100-200 samples** but **several hundred miRNA features** after QC filtering. This is a **$p \gg n$** problem: more features than samples. Our validation cohort GSE46579 is in this situation (273 miRNAs, 65 samples). GSE120584 is unusually large (1,296 AD + Control samples, ~700 features), so $n > p$ — but with a weak, diffuse signal, the risk of fitting noise remains, and feature selection still matters.

Why this is catastrophically bad for ML without feature selection:

Imagine trying to find the best fitting line through 2 data points — you can always find a *perfect* fit (the line passes through both points), but this fit tells you nothing about the true relationship. With 500 features and 150 samples, a classifier has enormous freedom to “memorize” the training data by exploiting random correlations that exist only in this particular sample. It will appear to perform perfectly during training but fail completely on new patients — a phenomenon called **overfitting**.

The mathematical intuition: in high dimensions, any two random samples will appear very similar to each other (curse of dimensionality), making it impossible to define meaningful decision boundaries.

Feature selection is not optional in biomarker ML — it is mandatory.

The goal: select the smallest set of miRNAs that contain the most discriminative biological information, discarding the hundreds of miRNAs that vary only due to noise.

4.3.2 Three Classes of Feature Selection

Feature selection methods fall into three categories based on how they interact with the learning algorithm:

ALL FEATURES (500 miRNAs)

|
v

Filter Methods (no ML model)	Wrapper Methods (use ML model)	Embedded Methods (inside ML)
Mann-Whitney U	Recursive Feature	LASSO (L1 reg)
t-test + FDR	Elimination (RFE)	Random Forest
Variance filter		importance

v	v	v
Fast, scalable	Computationally	Best for final
Good first pass	intensive, but	model integration;
Misses	captures feature	captures feature
interactions	interactions	interactions

4.3.3 Filter Methods: Univariate Statistical Tests

Filter methods score each feature independently using a statistical test, then select the top-ranked features. They are fast, interpretable, and agnostic to the downstream classifier.

Mann-Whitney U test (Wilcoxon rank-sum test): - Non-parametric test for difference in distribution between two groups - Does not assume normality — important because miRNA expression values, even after normalization, can be skewed - Tests: “Is the rank ordering of AD samples different from controls for this miRNA?” - Produces a p-value per miRNA; apply BH correction for FDR

Choosing a threshold: For feature selection (not publication reporting), a more lenient FDR threshold (e.g., $q < 0.20$ or even just top N by p-value) is sometimes appropriate. The goal here is to pass the most informative features to the ML model, not to make definitive biological claims. The ML model itself will further refine the selection.

Python code for univariate filter feature selection
(Run in Week4_ML_Classifier.ipynb)

```
import numpy as np
import pandas as pd
from scipy import stats
from statsmodels.stats.multitest import multipletests

# Load feature matrix and labels
# X: samples x miRNAs (loaded from DE results export)
# y: group labels (0 = Control, 1 = AD)

X = pd.read_csv("data/processed/GSE120584_expr_forML.csv", index_col=0)
meta = pd.read_csv("data/processed/GSE120584_metadata_clean.csv", index_col=0)

# Binary comparison: AD vs Control only
mask = meta["group"].isin(["Control", "Alzheimer's Disease"])
X_bin = X.loc[mask]
y_bin = (meta.loc[mask, "group"] == "Alzheimer's Disease").astype(int)

# Mann-Whitney U test for each miRNA
mw_pvalues = np.zeros(X_bin.shape[1])
for i, mirna in enumerate(X_bin.columns):
    group0 = X_bin.loc[y_bin == 0, mirna].values
    group1 = X_bin.loc[y_bin == 1, mirna].values
    _, mw_pvalues[i] = stats.mannwhitneyu(group0, group1, alternative="two-sided")

# Apply Benjamini-Hochberg correction
reject, padj, _, _ = multipletests(mw_pvalues, method="fdr_bh")

# Build ranked feature table
filter_results = pd.DataFrame({
    "miRNA": X_bin.columns,
```

```

    "pvalue": mw_pvalues,
    "padj": padj,
    "neg_log10_padj": -np.log10(padj + 1e-300),
    "significant_q05": reject
}).sort_values("pvalue")

print(f"Significant miRNAs (FDR < 0.05): {filter_results['significant_q05'].sum()}")
print("\nTop 20 miRNAs by Mann-Whitney U p-value:")
print(filter_results.head(20)[["miRNA", "pvalue", "padj"]])

```

4.3.4 Wrapper Methods: Recursive Feature Elimination (RFE)

RFE is an iterative approach: 1. Train a classifier on all features 2. Rank features by importance (using model coefficients or feature importances) 3. Remove the least important feature(s) 4. Repeat from step 1 until the desired number of features remains

The features that survive the longest (eliminated last) are the most important.

Advantage over filter methods: RFE captures feature *interactions* — it will keep two miRNAs that individually seem unimportant but together are highly discriminative.

Disadvantage: Computationally expensive (fits the model many times); can overfit if not properly cross-validated.

```

from sklearn.feature_selection import RFECV
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler

# Scale features (required for SVM-based RFE)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_bin)

# RFECV: RFE with cross-validated selection of optimal feature count
# Uses a linear SVM as the underlying estimator (fast, interpretable coefficients)
rfe = RFECV(
    estimator=SVC(kernel="linear", C=1.0),
    step=1,           # remove 1 feature per iteration
    cv=5,             # 5-fold cross-validation to determine optimal N
    scoring="roc_auc", # optimize for AUC
    n_jobs=-1         # use all available CPU cores
)

rfe.fit(X_scaled, y_bin)

print(f"Optimal number of features: {rfe.n_features_}")
selected_features_rfe = X_bin.columns[rfe.support_].tolist()
print("Selected features:")
print(selected_features_rfe)

```

4.3.5 Embedded Methods: LASSO Regularization

LASSO (Least Absolute Shrinkage and Selection Operator) is a regularized regression method that performs feature selection *during* model fitting. It adds a penalty term (L1 regularization) to the loss function that forces small coefficients to become exactly zero.

The result: A logistic regression model that uses only a subset of features — those with non-zero coefficients — to make predictions. The L1 penalty automatically selects the most informative features.

The regularization parameter C: In scikit-learn, $C = 1/\lambda$ where λ is the regularization strength. **Small C = strong regularization = fewer selected features.** The optimal C is found by cross-validated grid search.

```
from sklearn.linear_model import LogisticRegressionCV

# LASSO logistic regression with cross-validated regularization selection
lasso_cv = LogisticRegressionCV(
    Cs=np.logspace(-3, 1, 50), # test 50 values of C from 0.001 to 10
    cv=5,
    penalty="l1",
    solver="liblinear",
    scoring="roc_auc",
    max_iter=1000,
    random_state=42
)

lasso_cv.fit(X_scaled, y_bin)

# Features with non-zero coefficients are "selected"
coef = lasso_cv.coef_[0]
selected_lasso = X_bin.columns[coef != 0].tolist()
print(f"LASSO selected {len(selected_lasso)} features at optimal C = {lasso_cv.C_[0]:.4f}")

# Show selected features with their coefficients
lasso_df = pd.DataFrame({
    "miRNA": X_bin.columns[coef != 0],
    "coefficient": coef[coef != 0]
}).sort_values("coefficient", key=abs, ascending=False)
print(lasso_df)
```

4.3.6 Embedded Methods: Random Forest Feature Importance

Random forests compute feature importance as the **mean decrease in Gini impurity** — how much including each feature reduces uncertainty in class predictions across all decision trees. This is covered in detail in Module 4.6.

4.3.7 Choosing Among Feature Selection Methods

Method	When to Choose
Mann-Whitney U / t-test	Initial pass on a new dataset; biologically interpretable ranking; fast
LASSO	When you want the final model to also be logistic regression; forces sparsity naturally
RFE with SVM	When you want a rigorous wrapper method; computationally feasible for $N < 500$ features
Random Forest importance	When you plan to use RF as your final model; produces stability estimates across trees

Method	When to Choose
Combination (consensus)	For publication-quality biomarker panels: select miRNAs that appear in top lists from two or more methods

Best practice for biomarker discovery: Never rely on a single feature selection method. Use two to three methods and take the intersection or majority-vote consensus. miRNAs that appear as top features across multiple independent methods are far more likely to represent true biological signals than those selected by a single method.

MODULE 4.4 — Building a Logistic Regression Classifier

4.4.1 What is Logistic Regression?

Logistic regression is the foundational classification algorithm. Despite its name, it is a *classification* method (predicting a binary outcome: AD vs Control) rather than a regression method.

The core idea: 1. Compute a linear combination of features: $z = \text{beta}_0 + \text{beta}_1 * \text{miR-21} + \text{beta}_2 * \text{miR-29b} + \dots$ 2. Transform z through the logistic (sigmoid) function: $p = 1 / (1 + \exp(-z))$ 3. The result is a probability between 0 and 1: $p = \text{probability of AD}$

Input features (miRNA levels)

miR-21-5p: 8.3
miR-29b-3p: 6.1
miR-146a: 9.7
...

...

|
v

Linear combination: $z = \text{beta}_0 + \text{beta}_1(8.3) + \text{beta}_2(6.1) + \text{beta}_3(9.7) + \dots$

|
v

Sigmoid function: $p = 1 / (1 + \exp(-z))$

|
v

Output: $p = 0.78 \rightarrow \text{Classify as AD } (p > 0.5)$

$p = 0.23 \rightarrow \text{Classify as Control}$

Why start with logistic regression? It is fast to train, easy to interpret, and provides a baseline against which more complex models (SVM, RF) can be compared. If a more complex model does not substantially outperform logistic regression, the simpler model should be preferred (Occam's razor principle).

4.4.2 Train/Test Split: The Foundation of Honest Evaluation

Before any model is trained, you must split your data into a **training set** and a **test set**. The model is trained *only* on the training data and evaluated *only* on the test data. This simulates applying the model to new, unseen patients.

Why 80/20 split? With the 1,296 AD + Control samples of GSE120584, an 80/20 split gives: - Training set: ~1,037 samples — enough for stable model fitting - Test set: ~259 samples (about 202 AD and 57 Control) — enough for a reasonably precise performance estimate

In a typical cohort of ~150 samples the test set would hold only ~30 samples, which is why cross-validation (Section 4.7.5) matters so much for smaller studies.

Why stratified split? Ensures that both training and test sets have the same proportion of AD and Control samples. Without stratification, you might get (by chance) all AD samples in training and all Control in test — a useless split.

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (confusion_matrix, classification_report,
                             roc_auc_score, roc_curve)

import matplotlib.pyplot as plt
import seaborn as sns

# ---- Select features from DE analysis (top N from filter step) ----
# Use top 50 miRNAs by Mann-Whitney FDR as input to ML
top_features = filter_results.head(50)["miRNA"].tolist()
X_selected = X_bin[top_features]

# ---- Train/test split: 80% training, 20% test, stratified ----
X_train, X_test, y_train, y_test = train_test_split(
    X_selected, y_bin,
    test_size=0.20,
    stratify=y_bin,      # preserves class proportions in both splits
    random_state=42      # for reproducibility
)

print(f"Training set: {X_train.shape[0]} samples")
print(f"  AD: {y_train.sum()}, Control: {(y_train == 0).sum()}")
print(f"Test set: {X_test.shape[0]} samples")
print(f"  AD: {y_test.sum()}, Control: {(y_test == 0).sum()}")

# ---- Feature scaling: StandardScaler ----
# Logistic regression and SVM are sensitive to feature scale.
# StandardScaler transforms each feature to mean=0, std=1.
# IMPORTANT: Fit scaler ONLY on training data, then apply to both train and test.
# Fitting on the test set would cause "data leakage" -- the model would
# implicitly "see" test data during training.

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)  # fit AND transform training
X_test_scaled = scaler.transform(X_test)        # transform only (do NOT fit)
```

4.4.3 Fitting Logistic Regression and Interpreting Coefficients

```
# ---- Fit logistic regression with L2 regularization ----
# C parameter controls regularization strength: lower C = stronger regularization
# Start with C=1.0 (default); tune with cross-validation

lr = LogisticRegression(
    C=1.0,          # regularization parameter (1/lambda)
    penalty="l2",   # L2 (Ridge) regularization -- shrinks all coefficients but keeps all features
    solver="lbfgs", # optimization algorithm; lbfgs is robust for moderate-size problems
    max_iter=1000,  # maximum iterations for convergence
    random_state=42
)
```

```

lr.fit(X_train_scaled, y_train)

# ---- Interpret coefficients ----
# The coefficient beta for each miRNA is the change in log-odds of AD
# for a 1-standard-deviation increase in that miRNA's expression.
# Positive beta: higher expression -> higher probability of AD
# Negative beta: higher expression -> lower probability of AD (downregulated in AD)

coef_df = pd.DataFrame({
    "miRNA": top_features,
    "coefficient": lr.coef_[0],
    "abs_coefficient": np.abs(lr.coef_[0])
}).sort_values("abs_coefficient", ascending=False)

print("\nTop 15 miRNAs by logistic regression coefficient:")
print(coef_df.head(15)[["miRNA", "coefficient"]])

# Plot coefficient magnitudes (feature importance)
fig, ax = plt.subplots(figsize=(8, 6))
top15_coef = coef_df.head(15).sort_values("coefficient")
colors = ["#D73027" if c > 0 else "#4575B4" for c in top15_coef["coefficient"]]
ax.barh(top15_coef["miRNA"], top15_coef["coefficient"], color=colors, alpha=0.8)
ax.axvline(x=0, color="black", linewidth=0.8)
ax.set_xlabel("Log-Odds Coefficient (AD vs Control)")
ax.set_title("Logistic Regression: Top 15 Feature Coefficients", fontweight="bold")
plt.tight_layout()
plt.savefig("results/LR_coefficients.png", dpi=150)
plt.show()

```

Biological interpretation of a positive coefficient: A logistic regression coefficient of +0.85 for miR-146a means: *for each one-standard-deviation increase in miR-146a expression, the log-odds of being in the AD group increases by 0.85, corresponding to an odds ratio of $\exp(0.85) = 2.34$.* In biological terms: patients with higher circulating miR-146a have roughly 2.3-fold higher odds of having AD — consistent with the known role of miR-146a as a master regulator of neuroinflammation via the TLR/NF- κ B pathway.

4.4.4 L1 vs L2 Regularization

Feature	L1 (LASSO)	L2 (Ridge)
Penalty term	Sum of absolute values of coefficients	Sum of squared coefficients
Effect on coefficients	Forces some to exactly zero	Shrinks all but keeps all non-zero
Feature selection	Yes — sets irrelevant features to zero	No — keeps all features with small values
When to use	When you want sparse model (few features)	When many features contribute a little
Sklearn parameter	penalty="l1"	penalty="l2"

For biomarker panel discovery, **L1 (LASSO) is preferred** because it produces a sparse model with only the most informative miRNAs — exactly what we need for a clinical RT-qPCR panel. L2 is preferred when you believe all features contribute and want to reduce multicollinearity effects.

4.5.1 The Maximal Margin Intuition

Support Vector Machines answer this question in a principled way: **find the hyperplane that maximizes the margin between the two classes**. The margin is the width of the buffer zone between the decision boundary and the nearest samples from each class. The nearest samples — those that define the margin — are called **support vectors**.

Wider margin = more robust to noise = better generalization

4.5.2 The Kernel Trick for Non-Linear Separation

The most commonly used kernel for genomic data is the **Radial Basis Function (RBF) kernel**, also called the Gaussian kernel:

This kernel measures the similarity between two samples based on the Euclidean distance between their miRNA expression profiles, weighted by the gamma parameter. It effectively maps the data into an infinite-dimensional feature space where non-linear boundaries in the original space become linear.

4.5.3 Cross-Validated Grid Search for SVM Hyperparameters

```
from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV, StratifiedKFold
```

```

# Define the hyperparameter grid to search
param_grid = {
    "C": [0.01, 0.1, 1.0, 10.0, 100.0],
    "gamma": [0.001, 0.01, 0.1, 1.0, "scale"], # "scale" = 1/(n_features * X.var())
    "kernel": ["rbf"]
}

# 5-fold stratified cross-validation for hyperparameter selection
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

svm_grid = GridSearchCV(
    estimator=SVC(probability=True, # needed to get probability outputs for ROC curves
                  random_state=42),
    param_grid=param_grid,
    cv=cv,
    scoring="roc_auc", # optimize for AUC
    n_jobs=-1, # use all CPU cores
    verbose=1
)

svm_grid.fit(X_train_scaled, y_train)

print(f"\nBest SVM parameters: {svm_grid.best_params_}")
print(f"Best cross-validated AUC: {svm_grid.best_score_:.3f}")

# Final model with best parameters
best_svm = svm_grid.best_estimator_

```

4.5.4 Support Vectors and Their Biological Meaning

After fitting an SVM, you can identify the support vectors — the training samples that lie exactly on the margin boundaries. These samples are the most “borderline” cases in the dataset.

In miRNA biomarker terms, support vectors typically correspond to: - AD patients with atypically mild or early-stage disease - Control subjects with unusually high levels of AD-associated miRNAs (perhaps subclinical pathology or genetic risk)

Inspecting the clinical metadata of support vector samples can reveal biologically interesting patterns. For example, if many support vectors in the AD group are from patients at early MMSE stages, this suggests the classifier is capturing the transition zone of disease, consistent with the biology.

MODULE 4.6 — Random Forest

4.6.1 The Ensemble Principle

Random forest is an **ensemble method** — it combines many weak learners (individual decision trees) into one powerful classifier. The core insight: each individual decision tree is unreliable (high variance, prone to overfitting), but the *average* of many uncorrelated trees is stable and accurate.

Training Data
|

```

+---- Bootstrap sample 1 -> Decision Tree 1 -> Vote: AD
+---- Bootstrap sample 2 -> Decision Tree 2 -> Vote: Control
+---- Bootstrap sample 3 -> Decision Tree 3 -> Vote: AD
+---- Bootstrap sample 4 -> Decision Tree 4 -> Vote: AD
+----      ...
+---- Bootstrap sample N -> Decision Tree N -> Vote: AD
                                     |
                                     Majority vote: AD (4/5)
                                     -> Final prediction: AD

```

Bootstrap aggregation (bagging): Each tree is trained on a different bootstrap sample (random sampling with replacement) of the training data. Trees trained on different samples will make different errors, and averaging their predictions cancels out many of those errors.

Random feature subsets: At each split in each tree, only a random subset of features (typically $\sqrt{p}$ features) is considered. This *de-correlates* the trees — they cannot all rely on the same strong features, forcing them to explore different subsets of the feature space.

4.6.2 Gini Impurity and Node Splitting

Each decision tree splits the data at each node by finding the miRNA and threshold that best separates the classes. The measure of “best separation” is **Gini impurity**:

$$\text{Gini}(\text{node}) = 1 - \sum(\pi_i^2)$$

where π_i = fraction of samples of class i at this node

A pure node (all one class) has Gini = 0. A maximally impure node (50/50 split) has Gini = 0.5. The algorithm selects the split that maximally *reduces* Gini impurity.

Example:

Before split: 20 AD, 20 Control -> Gini = $1 - (0.5^2 + 0.5^2) = 0.50$ (high impurity)
 After split on miR-29b-3p expression > 7.2:
 Left child: 17 AD, 3 Control -> Gini = $1 - (0.85^2 + 0.15^2) = 0.255$
 Right child: 3 AD, 17 Control -> Gini = $1 - (0.15^2 + 0.85^2) = 0.255$
 Weighted average Gini: 0.255 -> large decrease from 0.50 -> good split!

4.6.3 Feature Importance via Mean Decrease in Gini Impurity

The **mean decrease in impurity (MDI)** importance of a feature is computed as: 1. For each tree, sum the decrease in Gini impurity across all splits where this feature is used 2. Weight by the fraction of samples reaching that node 3. Average across all trees in the forest

Higher importance means the feature is used more often in splits that effectively separate AD from Control across the forest.

```

from sklearn.ensemble import RandomForestClassifier
import matplotlib.pyplot as plt

```

```

# Fit Random Forest

```

```

rf = RandomForestClassifier(
    n_estimators=500,      # 500 trees; more is generally better but with diminishing returns
    max_features="sqrt",  # sqrt(n_features) considered at each split
    min_samples_leaf=2,   # minimum samples per leaf (prevents overfitting on small data)
    class_weight="balanced", # handles class imbalance
    random_state=42,

```

```

    n_jobs=-1
)

rf.fit(X_train_scaled, y_train)

# ---- Feature importances ----
importances = rf.feature_importances_
importance_df = pd.DataFrame({
    "miRNA": top_features,
    "importance": importances
}).sort_values("importance", ascending=False)

print("\nTop 20 miRNAs by Random Forest importance:")
print(importance_df.head(20))

# Plot top 20 feature importances
fig, ax = plt.subplots(figsize=(8, 7))
top20_imp = importance_df.head(20).sort_values("importance")
ax.barh(top20_imp["miRNA"], top20_imp["importance"],
        color="#4575B4", alpha=0.8, edgcolor="white")
ax.set_xlabel("Mean Decrease in Gini Impurity")
ax.set_title("Random Forest: Top 20 Feature Importances\n(AD vs Control, GSE120584)",
            fontweight="bold")
plt.tight_layout()
plt.savefig("results/RF_feature_importances.png", dpi=150)
plt.show()

```

4.6.4 Out-of-Bag (OOB) Error

A unique advantage of random forest: each tree is trained on approximately 63% of the data (bootstrap sample); the remaining ~37% of samples are “out-of-bag” (OOB) for that tree. We can evaluate each training sample using only the trees for which it was OOB — giving an unbiased estimate of generalization error *without* a separate test set.

```

# Enable OOB estimation
rf_oob = RandomForestClassifier(
    n_estimators=500,
    oob_score=True,      # enable OOB evaluation
    max_features="sqrt",
    class_weight="balanced",
    random_state=42,
    n_jobs=-1
)

rf_oob.fit(X_train_scaled, y_train)
print(f"OOB accuracy: {rf_oob.oob_score_:.3f}")
# Note: oob_score uses accuracy, not AUC.
# For imbalanced data, separately examine OOB confusion matrix.

```

4.6.5 Choosing the Number of Trees

```

# How AUC changes with number of trees
from sklearn.metrics import roc_auc_score

```

```

n_estimators_range = [10, 50, 100, 200, 300, 500, 750, 1000]
test_aucs = []

for n in n_estimators_range:
    rf_temp = RandomForestClassifier(
        n_estimators=n,
        max_features="sqrt", class_weight="balanced",
        random_state=42, n_jobs=-1
    )
    rf_temp.fit(X_train_scaled, y_train)
    auc = roc_auc_score(y_test, rf_temp.predict_proba(X_test_scaled)[:, 1])
    test_aucs.append(auc)

plt.figure(figsize=(7, 4))
plt.plot(n_estimators_range, test_aucs, marker="o", color="#4575B4", linewidth=2)
plt.xlabel("Number of Trees")
plt.ylabel("Test Set AUC")
plt.title("Random Forest: AUC vs Number of Trees", fontweight="bold")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("results/RF_n_estimators_curve.png", dpi=150)
plt.show()
# AUC should plateau; 500 trees is usually sufficient for datasets of this size

```

MODULE 4.7 — Model Evaluation

4.7.1 The Confusion Matrix

The confusion matrix is the fundamental output of binary classification evaluation. It cross-tabulates predicted labels against true labels:

	Predicted Control	Predicted AD
True Control	TN	FP
True AD	FN	TP

TN = True Negatives (correctly identified as no AD when no AD)

TP = True Positives (correctly identified as AD when AD)

FN = False Negatives (predicted no AD but actually AD -- missed case)

FP = False Positives (predicted AD but actually control -- false alarm)

Clinical interpretation: In an AD screening context, false negatives are more dangerous than false positives. A missed AD diagnosis means a patient goes untreated. A false positive leads to additional (but not harmful) confirmatory testing. This asymmetry influences which metric we optimize: we typically prioritize **sensitivity** (minimizing false negatives) over specificity.

```

from sklearn.metrics import (confusion_matrix, ConfusionMatrixDisplay,
                             roc_auc_score, roc_curve,
                             precision_recall_curve, average_precision_score)
import matplotlib.pyplot as plt

# Generate predictions from all three models
y_pred_lr = lr.predict(X_test_scaled)
y_pred_svm = best_svm.predict(X_test_scaled)
y_pred_rf = rf.predict(X_test_scaled)

```

```

y_prob_lr   = lr.predict_proba(X_test_scaled)[: , 1]
y_prob_svm  = best_svm.predict_proba(X_test_scaled)[: , 1]
y_prob_rf   = rf.predict_proba(X_test_scaled)[: , 1]

# Confusion matrices side by side
fig, axes = plt.subplots(1, 3, figsize=(14, 4))
for ax, y_pred, title in zip(
    axes,
    [y_pred_lr, y_pred_svm, y_pred_rf],
    ["Logistic Regression", "SVM (RBF)", "Random Forest"]
):
    cm = confusion_matrix(y_test, y_pred)
    disp = ConfusionMatrixDisplay(cm, display_labels=["Control", "AD"])
    disp.plot(ax=ax, colorbar=False, cmap="Blues")
    ax.set_title(title, fontweight="bold")
plt.suptitle("Confusion Matrices -- Test Set (GSE120584)", fontweight="bold")
plt.tight_layout()
plt.savefig("results/confusion_matrices.png", dpi=150)
plt.show()

```

4.7.2 Clinical Metrics from the Confusion Matrix

```

def clinical_metrics(y_true, y_pred, model_name):
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()

    sensitivity = tp / (tp + fn) if (tp + fn) > 0 else 0 # True Positive Rate / Recall
    specificity = tn / (tn + fp) if (tn + fp) > 0 else 0 # True Negative Rate
    ppv         = tp / (tp + fp) if (tp + fp) > 0 else 0 # Positive Predictive Value
    npv         = tn / (tn + fn) if (tn + fn) > 0 else 0 # Negative Predictive Value
    accuracy    = (tp + tn) / (tp + tn + fp + fn)

    print(f"\n{model_name}:")
    print(f"  Sensitivity (recall):    {sensitivity:.3f} -- proportion of AD correctly identified")
    print(f"  Specificity:             {specificity:.3f} -- proportion of Controls correctly identified")
    print(f"  PPV (precision):        {ppv:.3f} -- of those called AD, fraction truly AD")
    print(f"  NPV:                    {npv:.3f} -- of those called Control, fraction truly Control")
    print(f"  Accuracy:               {accuracy:.3f}")
    return {"Sensitivity": sensitivity, "Specificity": specificity,
            "PPV": ppv, "NPV": npv, "Accuracy": accuracy}

metrics_lr = clinical_metrics(y_test, y_pred_lr, "Logistic Regression")
metrics_svm = clinical_metrics(y_test, y_pred_svm, "SVM (RBF)")
metrics_rf = clinical_metrics(y_test, y_pred_rf, "Random Forest")

```

What each metric means clinically:

Metric	Formula	Clinical Meaning
Sensitivity	$TP / (TP + FN)$	Of all AD patients, what fraction does the test correctly identify? A test with sensitivity = 0.90 misses 10% of AD cases.
Specificity	$TN / (TN + FP)$	Of all non-AD individuals, what fraction does the test correctly classify? Low specificity floods clinics with false alarms.

Metric	Formula	Clinical Meaning
PPV	$TP / (TP + FP)$	If the test says "AD," what is the probability this person actually has AD? Critically dependent on disease prevalence.
NPV	$TN / (TN + FN)$	If the test says "no AD," what is the probability this person truly does not have AD?
Accuracy	$(TP + TN) / N$	Overall correct rate. Misleading with imbalanced classes.

The prevalence trap: PPV is profoundly affected by disease prevalence. A test with sensitivity = 0.90 and specificity = 0.90 applied to a population where AD prevalence is 5% will have PPV = only 32% — most positive results are false positives! This is why validating biomarkers in population-representative cohorts (not just matched case-control) is essential for clinical translation.

4.7.3 ROC Curve and AUC

The **Receiver Operating Characteristic (ROC) curve** plots sensitivity (y-axis) against 1-specificity (false positive rate, x-axis) across all possible classification thresholds. It shows the inherent trade-off between sensitivity and specificity.

The **Area Under the Curve (AUC)** summarizes the full ROC curve in a single number: - AUC = 0.50: random classifier (useless) - AUC = 0.70-0.80: moderate discriminative ability - AUC = 0.80-0.90: good discriminative ability (clinically useful) - AUC = 0.90-1.00: excellent (rare in complex diseases) - AUC = 1.00: perfect classifier (suspect overfitting)

ROC curves for all three models

```
fig, ax = plt.subplots(figsize=(7, 6))

for y_prob, label, color in [
    (y_prob_lr, "Logistic Regression", "#1B7837"),
    (y_prob_svm, "SVM (RBF)", "#762A83"),
    (y_prob_rf, "Random Forest", "#D73027")
]:
    fpr, tpr, _ = roc_curve(y_test, y_prob)
    auc = roc_auc_score(y_test, y_prob)
    ax.plot(fpr, tpr, linewidth=2, color=color, label=f"{label} (AUC = {auc:.3f})")

ax.plot([0, 1], [0, 1], "k--", linewidth=1, label="Random (AUC = 0.500)")
ax.set_xlabel("1 - Specificity (False Positive Rate)", fontsize=12)
ax.set_ylabel("Sensitivity (True Positive Rate)", fontsize=12)
ax.set_title("ROC Curves -- AD vs Control Classification\n(Test Set, GSE120584)",
             fontweight="bold")
ax.legend(loc="lower right", fontsize=10)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("results/ROC_curves.png", dpi=150)
plt.show()
```

4.7.4 Why AUC Alone Is Insufficient: The Imbalanced Class Problem

Our dataset from GSE120584 is **strongly imbalanced**: 1,009 AD vs 287 Control (78% AD), and only 32 MCI in the three-class problem. A classifier that calls everyone "AD" is already 78%

accurate. In real screening settings the imbalance usually runs the other way, with AD a small minority. Either way, understanding why accuracy and AUC can mislead is critical.

Scenario: 10 AD patients, 90 Controls (10% prevalence — realistic in a memory clinic).

A classifier that predicts “Control” for *everyone* achieves: - Accuracy: 90% <- looks great! - Sensitivity: 0% <- useless clinically - AUC: 0.50 <- correctly reveals the uselessness

But now consider a classifier with AUC = 0.85 and sensitivity = 0.70. It correctly identifies 7 of 10 AD patients. But with only 10 AD cases, the ROC curve is very unstable — the “true” AUC may be anywhere from 0.65 to 1.00 based on which 7 patients happen to fall in the test set. AUC estimates from imbalanced, small test sets have enormous confidence intervals that are rarely reported.

The precision-recall (PR) curve is more informative for imbalanced datasets:

```
# Precision-Recall curves
fig, ax = plt.subplots(figsize=(7, 6))

for y_prob, label, color in [
    (y_prob_lr, "Logistic Regression", "#1B7837"),
    (y_prob_svm, "SVM (RBF)", "#762A83"),
    (y_prob_rfc, "Random Forest", "#D73027")
]:
    precision, recall, _ = precision_recall_curve(y_test, y_prob)
    ap = average_precision_score(y_test, y_prob)
    ax.plot(recall, precision, linewidth=2, color=color,
            label=f"{label} (AP = {ap:.3f})")

# Baseline: random classifier performance = prevalence
prevalence = y_test.mean()
ax.axhline(y=prevalence, color="black", linestyle="--", linewidth=1,
          label=f"Random (AP = {prevalence:.3f})")

ax.set_xlabel("Recall (Sensitivity)", fontsize=12)
ax.set_ylabel("Precision (PPV)", fontsize=12)
ax.set_title("Precision-Recall Curves -- AD vs Control", fontweight="bold")
ax.legend(loc="upper right", fontsize=10)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("results/PRC_curves.png", dpi=150)
plt.show()
```

4.7.5 Cross-Validation for Unbiased AUC Estimates

In a typical cohort of ~150 samples, a test set of ~30 samples (from an 80/20 split) is too small to reliably estimate true AUC; even with GSE120584’s larger test set, a single split gives a noisier estimate than using all the data. **Cross-validation** uses all data for both training and testing by repeatedly splitting the data into folds:

```
from sklearn.model_selection import cross_val_score, StratifiedKFold
import numpy as np

# Define 5-fold stratified cross-validation
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Cross-validate each model
models = {
```

```

    "Logistic Regression": lr,
    "SVM (RBF)": best_svm,
    "Random Forest": rf
}

print("\n=== 5-Fold Cross-Validated AUC ===")
print(f"{'Model':<25} {'Mean AUC':>10} {'95% CI':>20}")
print("-" * 58)

for name, model in models.items():
    cv_scores = cross_val_score(
        model, X_selected, y_bin,
        cv=cv,
        scoring="roc_auc",
        n_jobs=-1
    )
    mean_auc = cv_scores.mean()
    # Approximate 95% CI using mean +/- 1.96 x SE
    se = cv_scores.std() / np.sqrt(len(cv_scores))
    ci_lo = mean_auc - 1.96 * se
    ci_hi = mean_auc + 1.96 * se
    print(f"{'name':<25} {'mean_auc':>10.3f} ({ci_lo:.3f} - {ci_hi:.3f})")

```

Important limitation: Cross-validation within a single dataset does not replace external validation on an independent cohort. All 5 folds come from the same study, same processing pipeline, same patient population. Week 5 will address external validation on GSE46579.

4.7.6 Model Calibration

A well-calibrated model is one where, among all samples for which the model predicts 70% probability of AD, approximately 70% actually have AD. Calibration matters clinically because physicians interpret model outputs as probabilities.

```

from sklearn.calibration import CalibrationDisplay

fig, ax = plt.subplots(figsize=(6, 6))
for model, y_prob, label, color in [
    (lr, y_prob_lr, "Logistic Regression", "#1B7837"),
    (best_svm, y_prob_svm, "SVM (RBF)", "#762A83"),
    (rf, y_prob_rf, "Random Forest", "#D73027")
]:
    CalibrationDisplay.from_predictions(
        y_test, y_prob,
        n_bins=5,
        ax=ax,
        name=label,
        color=color
    )

ax.set_title("Calibration Curves (Reliability Diagrams)", fontweight="bold")
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("results/calibration_curves.png", dpi=150)
plt.show()

```

```
# Note: Logistic regression is generally well-calibrated by default.  
# Random forests tend to be poorly calibrated (overconfident) and may benefit  
# from Platt scaling: CalibratedClassifierCV(rf, method="sigmoid")
```

BIOLOGICAL INTERPRETATION CALLOUT: When miR-29b is the Top Feature

Imagine your random forest model identifies **miR-29b-3p** as the most important feature for AD vs Control classification. How do you interpret and validate this computationally-derived finding?

Step 1 — Check the direction. Is miR-29b-3p downregulated in AD (negative logFC)? This is consistent with the literature: the miR-29 family is among the most extensively documented AD-associated miRNAs. Multiple studies (including Hebert et al. 2008, Shioya et al. 2010) showed reduced miR-29a/b in AD brain tissue.

Step 2 — Check the target biology. miR-29b-3p directly targets **BACE1** (beta-site APP cleaving enzyme 1), the enzyme that cleaves amyloid precursor protein (APP) to produce the amyloidogenic Abeta peptide. Reduced miR-29b leads to elevated BACE1, which leads to increased Abeta production, which leads to amyloid plaque formation. This is the most direct mechanistic link between a miRNA and the central pathological event in AD.

Step 3 — Check consistency across datasets. Does miR-29b-3p also appear in the top features from your DESeq2 analysis of GSE46579? (In our data it is not among the significant GSE120584 miRNAs at all — a reminder that textbook AD miRNAs do not always reappear in a new cohort.) Does it appear in published biomarker panels (Zhao et al. 2020: yes, miR-29 family members appear in their 12-miRNA signature)?

Step 4 — Assess feature importance stability. Run the random forest 100 times with different random seeds. If miR-29b-3p appears in the top 10 features in more than 80% of runs, it is a stable finding. If its rank fluctuates widely, it may be co-linear with another feature.

Step 5 — Propose wet-lab validation. The computational finding that miR-29b-3p discriminates AD from Control in serum is a hypothesis that demands experimental validation: - qRT-PCR validation in an independent cohort with BACE1 protein measurements - Correlation analysis: does lower serum miR-29b-3p correlate with higher BACE1 in paired serum/CSF or in post-mortem brain tissue? - Functional study: transfection of miR-29b mimics into neuronal cells and measurement of BACE1 protein and Abeta secretion

This is the **biology-first principle** in action: computational analysis generates testable hypotheses; experimental validation confirms them.

WEEK 4 LAB SESSIONS

Lab 4A: Differential Expression Analysis in R (90 min)

Objective: Run the complete DE pipeline on GSE120584 and GSE46579, producing ranked feature lists for ML input.

Prerequisites: data/processed/ must contain files from Weeks 2 and 3.

Tasks: 1. Open Week4_DE_FeatureSelection.R in RStudio 2. Run Sections 1 through 4 (load data; limma on GSE120584: AD vs Control, MCI vs Control, AD vs MCI) 3. Inspect the results tables — identify the top 10 upregulated and top 10 downregulated miRNAs for each comparison 4. Run Section 5 (volcano and MA plots) — save the plots and identify which labeled miRNAs you recognize from the Week 1 literature review 5. Run Section 6 (DESeq2 on GSE46579) — compare the top

20 DE miRNAs with your limma results 6. Run Section 7 (overlap analysis) — which miRNAs are consistently DE in both datasets? 7. Run Sections 8-10 (Mann-Whitney filter, consensus table, export)

Checkpoint questions before proceeding to Lab 4B: - What is the fold change (linear scale, not log2) for your top ranked miRNA? - How many miRNAs pass $FDR < 0.05$ AND $|\log_2FC| > 0.3$ in the GSE120584 AD vs Control comparison — and how many pass $FDR < 0.05$ alone? - How many miRNAs overlap between the limma (GSE120584) and DESeq2 (GSE46579) significant lists?

Lab 4B: ML Classifier in Python (90 min)

Objective: Build, compare, and evaluate three classifiers using features from Lab 4A.

File: Week4_ML_Classifier.ipynb (Jupyter Notebook)

Tasks: 1. Cell 1: Load exported feature matrix from data/processed/GSE120584_expr_forML.csv 2. Cells 2-3: Apply Mann-Whitney filter, inspect the ranked miRNA list 3. Cell 4: Perform 80/20 stratified train/test split — confirm class proportions 4. Cells 5-7: Fit logistic regression, SVM, and random forest 5. Cell 8: Generate confusion matrices for all three models 6. Cell 9: Plot ROC curves on the same axes — compare AUC values 7. Cell 10: Run 5-fold cross-validation, report mean AUC +/- 95% CI for each model 8. Cell 11: Extract and plot feature importances from random forest 9. Cell 12: Look up the top 3 features from your RF model in miRTarBase — what are their validated targets? Are any targets relevant to AD biology?

WEEK 4 ASSIGNMENTS

Required Reading

1. **Love MI, Huber W, Anders S (2014).** Moderated estimation of fold change and dispersion for RNA-seq data with DESeq2. *Genome Biology* 15:550. DOI: [10.1186/s13059-014-0550-8](https://doi.org/10.1186/s13059-014-0550-8) Focus on: Section on size factor estimation, the lfcShrink innovation, independent filtering explanation
2. **Ritchie ME et al. (2015).** limma powers differential expression analyses for RNA-sequencing and microarray studies. *Nucleic Acids Research* 43(7):e47. DOI: [10.1093/nar/gkv007](https://doi.org/10.1093/nar/gkv007) Focus on: voom transformation section, eBayes moderation, comparison with other methods
3. **Ludwig N et al. (2019).** Machine Learning to Detect Alzheimer's Disease from Circulating Non-coding RNAs. *Genomics Proteomics Bioinformatics* 17(4):430-440. DOI: [10.1016/j.gpb.2019.09.004](https://doi.org/10.1016/j.gpb.2019.09.004) Focus on: Feature selection pipeline, ML method comparison, biological interpretation of top features

Reflection Questions (Discuss in Week 5 opening session)

1. Your DESeq2 analysis identifies 45 miRNAs as significant at $FDR < 0.05$, but your limma analysis of the validation dataset identifies only 12 significant miRNAs. The overlap is 8 miRNAs. Which 8 miRNAs would you trust most for further validation, and why does the overlap matter more than the size of either individual list?
2. You build a random forest classifier that achieves $AUC = 0.92$ on the training set but $AUC = 0.71$ on the 20% held-out test set. What has happened, and what steps would you take to address it?
3. miR-146a-5p has $\log_2FC = 1.8$ ($FDR = 0.001$) in your DESeq2 analysis and is the second-most important feature in your random forest model. How would you design a 6-month

wet-lab experiment to determine whether this miRNA is a true biomarker or an artifact of the computational analysis?

4. A clinical collaborator asks you: “Your model achieves 87% accuracy — is it ready for clinical use?” How do you respond? What additional evidence would be needed before clinical deployment?

Practical Exercise

Compare the top 20 miRNAs from three methods: (1) DESeq2 ranking by padj, (2) Mann-Whitney U filter ranking, (3) Random Forest importance ranking. Create a table showing which miRNAs appear in all three lists, two of three, or only one. For the miRNAs appearing in all three lists, look each one up in miRTarBase and write one sentence about its biological relevance to AD. Save this table as results/consensus_features_Week4.csv.

WEEK 4 GLOSSARY

Term	Definition
Differential expression (DE)	Statistical identification of features (miRNAs) whose mean expression differs significantly between two or more biological groups
log2 fold change (logFC)	The difference between group means expressed on the log2 scale; logFC = 1.0 means the AD group has exactly twice the expression level of the control group
FDR (False Discovery Rate)	The expected proportion of significant results that are false positives; controlled by Benjamini-Hochberg correction of p-values
adj.P.Val / padj	FDR-adjusted p-value; the primary filtering threshold for differential expression significance
eBayes moderation	limma’s empirical Bayes method for estimating per-feature variances by sharing information across all features; improves power for small N
lfcShrink	DESeq2 function that uses an empirical Bayes prior to pull extreme (unreliable) fold changes from low-count features toward zero
Volcano plot	Scatter plot with log2 fold change on the x-axis and -log10(p-value) on the y-axis; simultaneously visualizes effect size and statistical significance
MA plot	Mean-versus-difference plot; x-axis = average expression (A), y-axis = log2 fold change (M); used to detect expression-level-dependent normalization biases
Dimensionality	The number of features (miRNAs) in a dataset; high dimensionality ($p \gg n$) causes overfitting if not addressed with feature selection
Overfitting	A model that learns noise in the training data rather than true biological patterns; shows high training performance but poor performance on new samples
Filter method	Feature selection that scores each feature independently using a statistical test, without involving the ML model
Wrapper method	Feature selection that iteratively fits the ML model on subsets of features (e.g., RFE) to find the optimal subset
Embedded method	Feature selection that occurs as part of the ML model training process (e.g., LASSO, random forest importances)

Term	Definition
LASSO	Least Absolute Shrinkage and Selection Operator; logistic regression with L1 regularization that drives uninformative feature coefficients to exactly zero
L1 / L2 regularization	Penalty terms added to the loss function to prevent overfitting; L1 produces sparse models (zero coefficients), L2 shrinks all coefficients
StandardScaler	scikit-learn preprocessing step that transforms each feature to mean = 0 and standard deviation = 1; mandatory before distance-based models (SVM, logistic regression)
Train/test split	Division of data into training set (model fitting) and test set (unbiased evaluation); the test set must not be seen during any training or hyperparameter tuning
Stratified split	A train/test or cross-validation split that preserves the class proportion in both subsets; important for imbalanced or small datasets
Support vectors	Training samples that lie closest to the SVM decision boundary and define the margin; the decision boundary depends only on these samples
Kernel trick	Mathematical technique that allows SVM to find non-linear decision boundaries by implicitly mapping data to a higher-dimensional feature space
RBF kernel	Radial Basis Function kernel; the most common SVM kernel for genomic data; measures similarity between samples based on Euclidean distance in feature space
Bootstrap aggregation (bagging)	Random forest technique of training each tree on a different bootstrap sample (random sample with replacement) to produce uncorrelated trees
Gini impurity	Measure of class heterogeneity at a decision tree node; used as splitting criterion in random forest; 0 = pure node, 0.5 = maximally mixed
Feature importance (MDI)	Random forest metric; total decrease in Gini impurity attributed to a feature, averaged across all trees
Out-of-bag (OOB) error	Random forest's built-in unbiased error estimate; each training sample is evaluated by the trees for which it was not used in training
Confusion matrix	Two-by-two table (for binary classification) cross-tabulating predicted vs true labels: TP, TN, FP, FN
Sensitivity	$TP / (TP + FN)$; proportion of true positive cases correctly identified; also called recall or true positive rate
Specificity	$TN / (TN + FP)$; proportion of true negative cases correctly identified; also called true negative rate
PPV (Positive Predictive Value)	$TP / (TP + FP)$; probability that a positive test result is a true positive; depends strongly on disease prevalence
NPV (Negative Predictive Value)	$TN / (TN + FN)$; probability that a negative test result is a true negative
ROC curve	Receiver Operating Characteristic curve; plots sensitivity vs 1-specificity across all classification thresholds
AUC	Area Under the ROC curve; summarizes overall discrimination ability; ranges from 0.5 (random) to 1.0 (perfect)
Precision-recall curve	Alternative to ROC curve; more informative for imbalanced datasets; plots PPV vs recall
Cross-validation	Model evaluation technique that repeatedly splits data into training/validation folds; provides more stable AUC estimates than a single test split

Term	Definition
Calibration	The correspondence between predicted probabilities and actual event rates; a well-calibrated model with 80% predicted probability should be correct approximately 80% of the time

KEY REFERENCES (Week 4)

1. Ludwig N et al. (2019). Machine Learning to Detect Alzheimer's Disease from Circulating Non-coding RNAs. *Genomics Proteomics Bioinformatics* 17(4):430-440. DOI: [10.1016/j.gpb.2019.09.004](https://doi.org/10.1016/j.gpb.2019.09.004)
2. Zhao X et al. (2020). A Machine Learning Approach to Identify a Circulating MicroRNA Signature for Alzheimer Disease. *J Appl Lab Med* 5(1):15-28. DOI: [10.1373/jalm.2019.029595](https://doi.org/10.1373/jalm.2019.029595)
3. Xu A et al. (2022). Alzheimer's Disease Diagnostics Using miRNA Biomarkers and Machine Learning. *J Alzheimers Dis* 86(2):841-859. DOI: [10.3233/JAD-215502](https://doi.org/10.3233/JAD-215502)
4. Love MI, Huber W, Anders S (2014). Moderated estimation of fold change and dispersion for RNA-seq data with DESeq2. *Genome Biology* 15:550. DOI: [10.1186/s13059-014-0550-8](https://doi.org/10.1186/s13059-014-0550-8)
5. Ritchie ME et al. (2015). limma powers differential expression analyses for RNA-sequencing and microarray studies. *Nucleic Acids Research* 43(7):e47. DOI: [10.1093/nar/gkv007](https://doi.org/10.1093/nar/gkv007)
6. Zhu A, Ibrahim JG, Love MI (2019). Heavy-tailed prior distributions for sequence count data: removing the noise and preserving large differences. *Bioinformatics* 35(12):2084-2092. DOI: [10.1093/bioinformatics/bty895](https://doi.org/10.1093/bioinformatics/bty895)
7. Breiman L (2001). Random Forests. *Machine Learning* 45:5-32. DOI: [10.1023/A:1010933404324](https://doi.org/10.1023/A:1010933404324)
8. Tibshirani R (1996). Regression Shrinkage and Selection via the Lasso. *Journal of the Royal Statistical Society: Series B* 58(1):267-288. DOI: [10.1111/j.2517-6161.1996.tb02080.x](https://doi.org/10.1111/j.2517-6161.1996.tb02080.x)
9. Cortes C, Vapnik V (1995). Support-vector networks. *Machine Learning* 20:273-297. DOI: [10.1007/BF00994018](https://doi.org/10.1007/BF00994018)
10. Fawcett T (2006). An introduction to ROC analysis. *Pattern Recognition Letters* 27(8):861-874. DOI: [10.1016/j.patrec.2005.10.010](https://doi.org/10.1016/j.patrec.2005.10.010)
11. Benjamini Y, Hochberg Y (1995). Controlling the false discovery rate: a practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B* 57(1):289-300. DOI: [10.1111/j.2517-6161.1995.tb02031.x](https://doi.org/10.1111/j.2517-6161.1995.tb02031.x)
12. Saito T, Rehmsmeier M (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE* 10(3):e0118432. DOI: [10.1371/journal.pone.0118432](https://doi.org/10.1371/journal.pone.0118432)

NEXT WEEK PREVIEW: Advanced ML & Validation (Week 5)

Week 5 builds directly on this week's models. Key topics: - **Nested cross-validation:** Properly unbiased performance estimation with hyperparameter tuning inside cross-validation loops — the method required to avoid the most common overfitting mistake in biomarker ML - **External validation:** Applying the classifiers trained on GSE120584 directly to the independent cohort GSE46579 — the acid test of generalizability - **Ensemble methods:** Combining logistic regression, SVM, and random forest predictions into a consensus classifier (stacking, voting); why ensembles outperform individual models - **SHAP values:** Shapley Additive exPlanations for interpreting

individual-sample predictions from any model — which miRNAs drove this particular patient's classification? - **Three-class classification:** Extending binary AD vs Control models to discriminate AD, MCI, and Control simultaneously using multiclass SVM and multinomial logistic regression - **Introduction to deep learning:** Why a simple two-layer neural network applied to 500 miRNAs requires special regularization and when (not) to use it

Prepare: Before Week 5, ensure you have saved results/consensus_features_Week4.csv from the practical exercise. This feature list will be used as the starting point for Week 5's nested cross-validation.